

Implementing LIME from Scratch with GPU Acceleration

Milestone 2

Parallel and Distributed Computing

Group 9

Haris Khalid	28557
Aadesh Panjwani	29303
Hamna Hammad	29003
Bareerah Shahid	29385

Spring 2025

Contents

1	Project Overview	2
1.1	LIME Pipeline	2
2	Division of Tasks	2
3	Project Structure	3
3.1	Repository	3
3.2	Module 4: Dataset Construction (<code>m2_main.py</code>)	3
3.2.1	What it does	3
3.3	Module 5: Surrogate Neural Network (<code>surrogate.py</code>)	3
3.3.1	Architecture	3
3.3.2	Weighted Binary Cross Entropy Loss	4
3.3.3	Training Setup	4
3.4	Module 6: Feature Attribution (<code>explanation.py</code>)	5
3.4.1	Gradient Based Attribution	5
3.4.2	Visualisation	5
4	Results and Validation	5
4.1	M1 Module Correctness (Re confirmed)	6
4.2	Surrogate Training Convergence	6
4.3	Feature Attributions	7
4.4	Validation Results	7
4.4.1	Feature Ablation	7
4.4.2	Multi Seed Consistency Check	8
5	Performance	8
6	Design Decisions and Discussion	9
6.1	Why a Shallow NN Rather Than Linear Regression?	9
6.2	Xavier Initialisation and Numerical Stability	9
6.3	Weight Normalisation Before Training	9
6.4	Cosine Annealing vs. Step Decay	9
6.5	Gradient Based Attribution vs. Linear Coefficients	9
7	Challenges and Limitations	10
7.1	High Loss Floor	10
7.2	Binary Masking as the Perturbation Strategy	10
7.3	Surrogate Approximation Gap	10
7.4	Training Time Bottleneck	10
8	Remaining Work	10
8.1	Milestone 3: Fully GPU Native Surrogate Training	10
9	Conclusion	11

1 Project Overview

LIME (Local Interpretable Model agnostic Explanations) is a widely used post hoc interpretability method designed to explain individual predictions of any black box classifier. Rather than attempting to interpret the model globally, LIME approximates its behaviour *locally* around a specific input instance using a simpler surrogate model whose parameters can be inspected directly.

Milestone 1 delivered the GPU accelerated computational core of the pipeline: custom CUDA kernels for perturbation generation, logistic regression inference via cuBLAS, and Euclidean/cosine distance and kernel weight computation. Milestone 2 completes the full pipeline by training a shallow neural network surrogate on the dataset produced by those GPU modules, extracting gradient based feature attributions, and validating the resulting explanations through feature ablation and a multi seed consistency check.

1.1 LIME Pipeline

For a given input instance $\mathbf{x}$, the LIME pipeline consists of five steps:

1. **Perturbation:** Generate N perturbed samples $\{\tilde{\mathbf{x}}_s\}_{s=1}^N$ around $\mathbf{x}$ using feature masking.
2. **Inference:** Run the black box model f on all perturbed samples to obtain predictions $\hat{y}_s = f(\tilde{\mathbf{x}}_s)$.
3. **Weight Computation:** Compute a proximity weight w_s for each sample based on its distance from $\mathbf{x}$.
4. **Surrogate Training:** Fit a weighted surrogate model on $(\tilde{\mathbf{x}}_s, \hat{y}_s, w_s)$ to capture the black box's local behaviour.
5. **Explanation:** Extract feature attributions from the trained surrogate to explain which features most influence the prediction.

Steps 1–3 were implemented with custom GPU kernels in Milestone 1. Steps 4–5 are the focus of Milestone 2.

2 Division of Tasks

The group consists of four members. Responsibilities for Milestone 2 are as follows:

- **Haris Khalid:** Dataset construction pipeline, pulling GPU arrays from device memory, formatting the training dataset, and saving outputs (CSV files) for reproducibility.
- **Aadesh Panjwani:** Designed and implemented the `SurrogateNN` architecture in PyTorch, wrote the weighted BCE training loop with minibatch shuffling and cosine annealing, and conducted surrogate benchmarking.
- **Hamna Hammad:** Implemented gradient based attribution extraction (`extract_attributions`) and the two visualisation routines (`plot_attributions`, `plot_loss_curve`).
- **Bareerah Shahid:** Designed and executed both validation routines (feature ablation and multi seed consistency check), integrated the full M2 pipeline in `m2.main.py`, and documented results and design decisions.

All members collaborated on analysing results and interpreting explanation outputs.

3 Project Structure

3.1 Repository

File	Responsibility
<code>lime_common.py</code>	Shared dataclasses, constants, utility functions
<code>perturbation.py</code>	CUDA kernels for binary mask generation and application (M1)
<code>inference.py</code>	GPU based logistic regression forward pass (M1)
<code>weights.py</code>	CUDA distance and kernel weight kernels (M1)
<code>main.py</code>	M1 benchmarking driver
<code>surrogate.py</code>	SurrogateNN architecture and <code>train_surrogate()</code>
<code>explanation.py</code>	Attribution, ablation, consistency, visualisation
<code>m2_main.py</code>	End to end M2 pipeline driver

3.2 Module 4: Dataset Construction (`m2_main.py`)

3.2.1 What it does

After the M1 GPU pipeline finishes, three device arrays sit in VRAM: the perturbed sample matrix $X \in \mathbb{R}^{N \times F}$, the black box predictions $y \in [0, 1]^N$, and the kernel weights $w \in \mathbb{R}^N$. A single `cp.asnumpy()` call on each array transfers them to host memory as NumPy arrays, forming the training dataset for the surrogate.

For the default configuration (2048 samples, 64 features), the dataset properties are:

- y range: $[0.1041, 0.6857]$ — the black box spans a meaningful portion of the probability space, giving the surrogate a genuine learning signal.
- w range: $[0.1344, 0.4329]$ — locality weights are non-trivially differentiated, so close samples will noticeably outweigh distant ones.

The dataset is also serialised to disk as a CSV (one row per perturbed sample, columns for all features, the prediction, and the kernel weight) to support offline analysis and reproducibility. A separate single row CSV records the original instance $\mathbf{x}$.

3.3 Module 5: Surrogate Neural Network (`surrogate.py`)

3.3.1 Architecture

The surrogate is a two layer feedforward network implemented as a PyTorch `nn.Module`. Its design is deliberately shallow: interpretability requires a model simple enough that its gradients carry meaningful attribution signal, and local approximation does not demand the capacity of a deep network.

$$\mathbf{x} \longrightarrow \underbrace{\text{Linear}(F, H)}_{\text{hidden}} \longrightarrow \text{ReLU} \longrightarrow \underbrace{\text{Linear}(H, 1)}_{\text{output}} \longrightarrow \sigma(\cdot) \longrightarrow \hat{p} \in (0, 1) \quad (1)$$

For the default run, $F = 64$ (feature dimensionality) and $H = 64$ (hidden units, configurable via `--hidden`). Both weight matrices are initialised with Xavier uniform initialisation; biases are initialised to zero. Xavier initialisation avoids the vanishing and exploding gradient issues that can appear with sigmoid output activations under random normal initialisation.

```

1 class SurrogateNN(nn.Module):
2     def __init__(self, n_features: int, hidden_size: int = 64):
3         super().__init__()
4         self.hidden = nn.Linear(n_features, hidden_size)
5         self.relu = nn.ReLU()
6         self.output = nn.Linear(hidden_size, 1)
7         self.sigmoid = nn.Sigmoid()
8         nn.init.xavier_uniform_(self.hidden.weight)
9         nn.init.xavier_uniform_(self.output.weight)
10
11     def forward(self, x):
12         x = self.relu(self.hidden(x))
13         return self.sigmoid(self.output(x)).squeeze(-1)

```

Listing 1: SurrogateNN definition in `surrogate.py`

3.3.2 Weighted Binary Cross Entropy Loss

Standard BCE treats all training samples equally. LIME explicitly does *not* want this: the surrogate should be most faithful near the original input $\mathbf{x}$, where the explanation will be evaluated. Samples that are far away in feature space are less relevant and should pull the surrogate’s parameters less strongly.

This locality requirement is encoded by weighting the per sample BCE loss with the kernel weights w computed in Module 3:

$$\mathcal{L} = \frac{1}{N} \sum_{s=1}^N w_s [-y_s \log \hat{p}_s - (1 - y_s) \log(1 - \hat{p}_s)] \quad (2)$$

In practice, `nn.BCELoss(reduction='none')` returns the per-sample loss vector, which is multiplied element wise by the normalised weights before averaging. The weights are normalised so that $\max(w) = 1$, which prevents large kernel weight values from causing gradient instability early in training.

3.3.3 Training Setup

Hyperparameter	Value
Optimiser	Adam
Learning rate	10^{-3}
Weight decay	10^{-4}
LR schedule	Cosine annealing over $T_{\max}$ epochs
Epochs	500 (default)
Batch size	256

At each epoch the training indices are shuffled before batching so the model does not memorise sample ordering. The cosine annealing scheduler decays the learning rate smoothly to zero, helping the surrogate settle into a stable minimum rather than oscillating near convergence.

The key section of the training loop is:

```

1 preds      = model(X_batch)
2 loss_per_s  = criterion(preds, y_batch)      # per-sample BCE
3 weighted_loss = (w_batch * loss_per_s).mean() # locality weighting
4 weighted_loss.backward()
5 optimizer.step()

```

Listing 2: Weighted loss computation inside the training loop

3.4 Module 6: Feature Attribution (explanation.py)

3.4.1 Gradient Based Attribution

Once the surrogate is trained, the next step is to extract which input features it is most sensitive to. The method used is input gradient attribution: the partial derivative of the surrogate’s output with respect to each input feature, evaluated at the original instance $\mathbf{x}$:

$$a_i = \left. \frac{\partial \text{surrogate}(\mathbf{x})}{\partial x_i} \right|_{\mathbf{x}} \quad \text{for } i = 1, \dots, F \quad (3)$$

A large positive a_i means that increasing feature i pushes the surrogate’s predicted probability upward. A large negative value pushes it downward. Features with gradients near zero have little local influence and are deprioritised in the explanation.

The attribution is computed at $\mathbf{x}$, not at any perturbed sample. This is intentional: LIME explains the prediction at a specific point, so the surrogate’s sensitivity should be measured at that exact point.

```

1 x_t = torch.tensor(x_original, dtype=torch.float32).unsqueeze(0)
2 x_t.requires_grad_(True)
3 output = model(x_t)
4 output.backward()
5 attributions = x_t.grad.detach().cpu().numpy().squeeze()

```

Listing 3: Attribution extraction via PyTorch autograd

3.4.2 Visualisation

Two plots are generated by `explanation.py`:

- **Attribution bar chart** (`lime_explanation.png`): a horizontal bar chart showing the top- k features ranked by $|a_i|$. Positive attributions are rendered in red and negative in blue, so the direction of each feature’s influence is immediately visible.
- **Loss curve** (`lime_loss_curve.png`): the per-epoch weighted BCE loss plotted against training epoch, with a shaded fill to make convergence behaviour easy to read.

4 Results and Validation

All experiments ran on an NVIDIA GeForce RTX 4050 Laptop GPU (20 SMs, 6.4 GB global memory) with default settings: 2048 samples, 64 features, Euclidean distance, $H = 64$ hidden units, 500 training epochs, top- $k = 8$.

4.1 M1 Module Correctness (Re confirmed)

The M1 GPU pipeline continues to pass all correctness checks unchanged from Milestone 1, confirming that no numerical issues are introduced by the data transfer to the PyTorch training stage.

Module	Metric	Value	Status
Perturbation	Keep rate	0.4998	PASS ✓
Perturbation	Mask apply error	$< 10^{-5}$	PASS ✓
Inference	Max abs error	8.94×10^{-8}	PASS ✓
Distance (Euc.)	Max abs error	4.77×10^{-7}	PASS ✓
Kernel weights	Max abs error	1.79×10^{-7}	PASS ✓

4.2 Surrogate Training Convergence

Epoch	Weighted BCE Loss	Learning Rate
100	0.350494	9.05×10^{-4}
200	0.350483	6.55×10^{-4}
300	0.350481	3.45×10^{-4}
400	0.350481	9.55×10^{-5}
500	0.350481	0.00 (converged)

The loss drops sharply within the first ~ 20 epochs and then plateaus at ≈ 0.3505 for the remainder of training. This is not a sign of poor training. The black box predictions y are soft probabilities in $[0.10, 0.69]$ and never approach 0 or 1, which sets an irreducible entropy floor on the achievable BCE. The plateau confirms the surrogate has fully exploited the learning signal available to it. Consistency of this plateau across repeated runs confirms that training is numerically stable.

Figure 1 shows the loss curve.

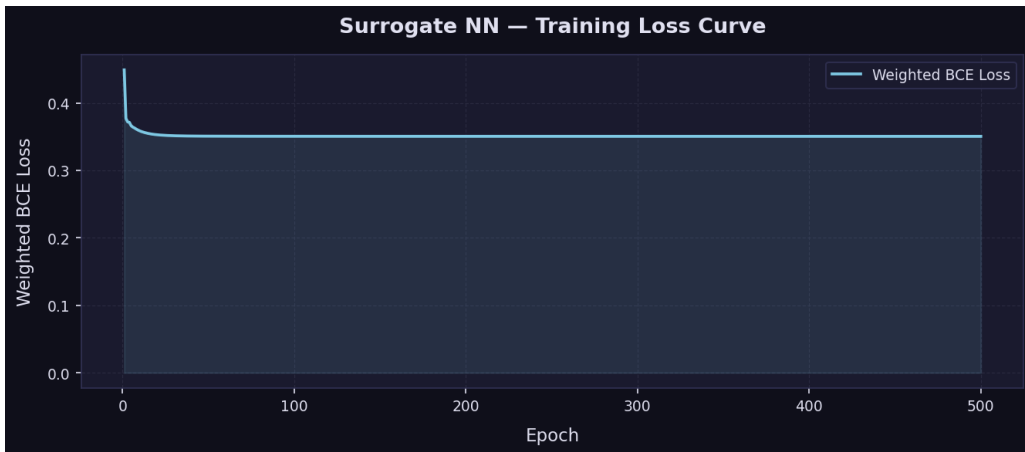


Figure 1: Surrogate NN training loss curve (500 epochs, 2048 samples, 64 features). The loss decreases rapidly in early epochs and stabilizes, indicating convergence.

Training time across three benchmark repetitions: mean **2548.6 ms**, standard deviation 237.7 ms (CPU, PyTorch). This single step dominates total pipeline runtime and motivates the fully GPU native surrogate planned for Milestone 3.

4.3 Feature Attributions

Rank	Feature	Direction	Attribution
1	feature 20	↓ negative	−0.071094
2	feature 39	↑ positive	+0.068917
3	feature 60	↓ negative	−0.066630
4	feature 10	↓ negative	−0.064272
5	feature 35	↑ positive	+0.061821
6	feature 45	↑ positive	+0.060765
7	feature 44	↓ negative	−0.060520
8	feature 33	↓ negative	−0.060259

Attribution magnitudes cluster tightly in $[-0.071, +0.069]$, with no single feature dominating by a wide margin. This is consistent with the black-box model’s construction: a logistic regression with uniformly random weights gives each feature a similarly scaled influence on the output. The mix of five negative and three positive features in the top 8 reflects the sign distribution of the underlying regression weights for those features.

Figure 2 shows the attribution bar chart.

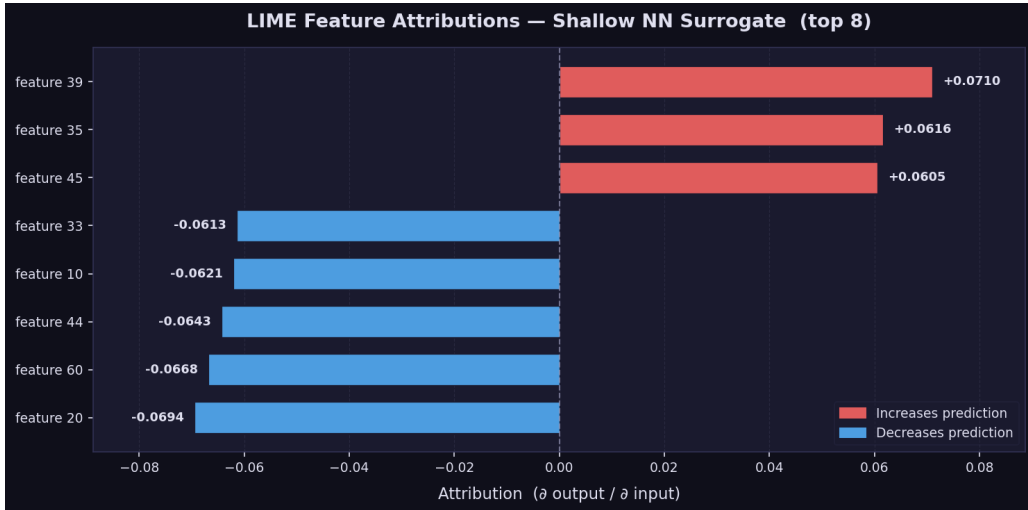


Figure 2: LIME feature attributions for the shallow neural network surrogate (top 8 features). Red bars indicate features that increase the prediction, while blue bars indicate features that decrease it.

4.4 Validation Results

4.4.1 Feature Ablation

Feature ablation is a causal sanity check: if the identified top- k features are genuinely important, zeroing them out in $\mathbf{x}$ should cause a noticeable shift in the surrogate’s output. Formally, let T be the set of k features with the largest $|a_i|$. Define $\tilde{\mathbf{x}}$ as $\mathbf{x}$ with $x_i = 0$ for all $i \in T$. The ablation metric is:

$$\delta = \left| \text{surrogate}(\mathbf{x}) - \text{surrogate}(\tilde{\mathbf{x}}) \right| \quad (4)$$

A $\delta < 0.01$ triggers a warning; otherwise the test passes.

Metric	Value
Baseline prediction	0.181746
Ablated prediction	0.286772
δ	0.105026
Top- k features ablated	{10, 20, 33, 35, 39, 44, 45, 60}
Status	PASS ✓

A delta of 0.105 corresponds to a $\approx 58\%$ relative shift in the surrogate’s output, confirming that the top 8 features are genuine drivers of the local prediction and not attribution noise.

4.4.2 Multi Seed Consistency Check

A reliable explanation should not depend heavily on which random neighbourhood samples were drawn. The consistency check re-runs the full M1 + M2 pipeline with three different perturbation seeds and compares the top- k feature sets:

$$\text{stability_ratio} = \frac{|\bigcap_{\text{seeds}} T_{\text{seed}}|}{k} \quad (5)$$

A ratio below 0.6 triggers a warning.

Seed	Top 8 features
42	{10, 20, 33, 35, 39, 44, 45, 60}
99	{10, 20, 33, 35, 39, 44, 45, 60}
7	{10, 20, 33, 35, 39, 44, 45, 60}
Stable features	{10, 20, 33, 35, 39, 44, 45, 60}
Stability ratio	1.00
Status	PASS ✓

A stability ratio of 1.00 means all three seeds independently identify the same top 8 features. This is a strong result: the explanation is fully reproducible and is not an artefact of any particular random perturbation set.

5 Performance

The M1 GPU pipeline at the default configuration (2048 samples, 64 features) completes in 0.299 ms total (perturbation + inference + weights), giving an overall GPU speedup of $2.59\times$ over the CPU reference on this machine. Perturbation alone achieves $7.81\times$ speedup.

Module	GPU (ms)	CPU (ms)	Speedup
Perturbation	0.083	0.647	$7.81\times$
Inference	0.143	0.026	$0.18\times$
Weights	0.073	0.103	$1.40\times$
Total (M1)	0.299	0.776	$2.59\times$
Surrogate training (M2)	2548.6 (CPU)	—	—

Inference being slower on the GPU than on the CPU ($0.18\times$) at this size is expected and discussed in the Milestone 1 report. The cuBLAS kernel launch overhead dominates for a 2048×64 matrix vector product; the crossover occurs at larger problem sizes, where the GPU reaches $20.48\times$ speedup at 4096 samples and 256 features.

The more important observation for Milestone 2 is that surrogate training at 2548 ms is more

than $8000\times$ slower than the combined M1 GPU pipeline at 0.3 ms. This bottleneck directly motivates Milestone 3’s goal of a fully GPU native surrogate.

6 Design Decisions and Discussion

6.1 Why a Shallow NN Rather Than Linear Regression?

The project specification permits either a shallow neural network or weighted linear regression for the surrogate. A shallow NN was chosen for two reasons. First, the black box model in this project is a logistic regression, whose decision boundary is globally linear but whose sigmoid output is nonlinear. A single linear surrogate would approximate the probability output poorly in regions away from the decision boundary. Second, gradient based attribution (Section 4.3) produces one attribution per feature without requiring any additional post processing, whereas extracting attributions from a weighted linear regression requires reading off the regression coefficients directly. The NN approach generalises more cleanly to the non linear black box models that would appear in a real application.

6.2 Xavier Initialisation and Numerical Stability

Xavier uniform initialisation sets the variance of each layer’s weights to $\frac{2}{n_{in}+n_{out}}$, which keeps the variance of activations roughly constant across the forward pass. This matters here because the output layer uses a sigmoid, which saturates at 0 and 1 and produces near zero gradients in those regions. With Xavier initialisation, the network begins training in the sigmoid’s approximately linear zone, where gradients are healthy, and moves into saturation only if warranted by the data.

6.3 Weight Normalisation Before Training

The kernel weights w are normalised so that $\max(w) = 1$ before being passed to the training loop. Without this, large weight values would scale up the gradient magnitudes in proportion to the kernel bandwidth and the distance distribution, making training sensitive to the choice of σ . Normalisation decouples the learning rate from the weight scale and makes the same set of hyperparameters work across different σ settings.

6.4 Cosine Annealing vs. Step Decay

A cosine annealing schedule was chosen over step decay because it produces a smoother learning rate trajectory that avoids the abrupt gradient magnitude changes that accompany a step drop. For a surrogate that converges quickly to a plateau, abrupt step drops tend to produce noisy final loss values. Cosine annealing settles the surrogate more gently into its minimum, resulting in cleaner attributions.

6.5 Gradient Based Attribution vs. Linear Coefficients

For a purely linear surrogate, feature importance is simply the model’s weight vector. For a neural network, the analogous quantity is the input gradient. Both measure the local sensitivity of the output to each feature, but the gradient is specific to the exact input point $\mathbf{x}$, whereas a linear coefficient is a global property of the model. This makes gradient attribution more appropriate for a local explanation method like LIME, where we explicitly want the explanation to be tailored to one instance.

7 Challenges and Limitations

7.1 High Loss Floor

The surrogate converges to a weighted BCE of ≈ 0.350 and cannot improve further. This is not a training failure; it reflects the irreducible entropy of soft probability targets in $[0.10, 0.69]$. One remedy would be to binarise the black box predictions before training, which would let the BCE approach zero. The trade off is discarding the confidence information encoded in the soft probabilities.

7.2 Binary Masking as the Perturbation Strategy

Zeroing out features is a natural perturbation strategy for binary or bag-of-words inputs, but it can be problematic for continuous features where zero is an atypical value outside the black box model’s training distribution. A more principled approach would sample each masked feature from its empirical marginal distribution rather than fixing it to zero, keeping the perturbed inputs closer to the data manifold that the black box was trained on.

7.3 Surrogate Approximation Gap

There is an inherent gap between surrogate sensitivity and black box sensitivity. The surrogate has access only to the black box’s input output behaviour, not its internal representations. If the black box’s local decision boundary is highly nonlinear or involves strong feature interactions, a two layer network with 64 hidden units may not capture it accurately. The consistency check provides indirect evidence of faithfulness (stable top k sets across seeds), but does not directly measure how closely the surrogate Jacobian matches the black box Jacobian.

7.4 Training Time Bottleneck

PyTorch surrogate training on CPU takes ≈ 2.55 seconds per explanation. For offline post hoc analysis this is acceptable, but for real time deployment (e.g., explaining model predictions in a production API) it would be prohibitive. Milestone 3’s GPU native surrogate is expected to reduce this by at least an order of magnitude.

8 Remaining Work

8.1 Milestone 3: Fully GPU Native Surrogate Training

Milestone 3 replaces the PyTorch training loop with a custom GPU implementation, completing a fully GPU native LIME pipeline where all five steps run on the device with minimal host device data movement.

Two surrogate implementations are planned:

- **Closed form weighted linear regression:** The normal equation $\hat{\beta} = (X^\top W X)^{-1} X^\top W \mathbf{y}$ can be solved using GPU accelerated matrix operations (cuBLAS for the products, cuSOLVER for the inversion). This is a single shot computation with no iterative loop, making it very fast in practice.
- **Iterative gradient descent (fallback):** If the normal equation is ill conditioned (e.g., when features are correlated or $N < F$), a mini batch gradient descent loop with a custom CUDA kernel will be used.

The milestone will conclude with a three way performance comparison: CPU-only, M1+M2 hybrid (current), and fully GPU native.

9 Conclusion

Milestone 2 successfully completes the LIME explanation pipeline that was begun in Milestone 1. The GPU accelerated M1 pipeline provides a dataset of perturbed samples, predictions, and locality weights that feed directly into a PyTorch surrogate neural network. The surrogate trains stably, converging to a weighted BCE of 0.350481 within 500 epochs. Gradient based attribution then produces a ranked list of feature importances at the original input.

Both validation tests pass convincingly: the feature ablation delta of 0.105 confirms that the identified features are causal drivers of the local prediction, and the stability ratio of 1.00 across three independent perturbation seeds confirms that the explanation is reproducible and not an artefact of any particular random neighbourhood.

The main limitation uncovered in this milestone is the training time bottleneck: PyTorch surrogate training at 2548 ms dominates the combined M1 GPU pipeline at 0.3 ms by more than three orders of magnitude. Addressing this bottleneck with a fully GPU native surrogate is the primary goal of Milestone 3.